

数据库系统原理

课设系统实现总结报告

林子淇，邓少儒，琚长昊

目录

一、 实现环境	1
1.1 系统开发环境	1
1.2 运行环境	1
1.3 技术栈架构	1
二、 系统功能结构	2
三、 基本表定义与完整性约束	3
3.1 基本表定义	3
3.2 完整性约束定义	3
3.3 索引定义	4
四、 系统安全性设计	4
4.1 用户认证与会话管理	4
4.2 外模式与权限控制	4
4.3 审计与追踪	4
五、 应用层数据处理逻辑	5
5.1 数据提交过程	5
5.2 自动账户生成逻辑	5
六、 主要技术与主要模块	6
6.1 主要技术	6
6.2 主要模块论述	6
七、 系统功能运行实例	7
7.1 登录与角色分流	7
7.2 数据录入与提交	7
7.3 自动排课流程	7
八、 源程序简要说明	9
九、 收获和体会	10

一、实现环境

本系统的设计与实现采用了针对 Windows 桌面环境优化的应用架构, 代码托管于 GitHub 平台, 并通过 GitHub Actions 实现了自动化的构建与发布流程。后端通过 Rust 语言保障高性能与内存安全, 前端结合 Vue 3 生态提供流畅的交互体验, 底层数据存储采用了嵌入式关系数据库 SQLite。

1.1 系统开发环境

开发工作主要在 Windows 11 操作系统下进行。开发工具链包括 Visual Studio Code 作为主要集成开发环境, Git 用于版本控制。构建环境依赖于 Node.js (v18+) 用于前端构建, Rust (v1.70+) 用于后端编译, 以及 Python (v3.11+) 用于约束求解器的开发与打包。

1.2 运行环境

系统目标运行平台为 Windows 10 及以上版本的桌面环境。由于采用了 Tauri 框架, 应用程序被编译为原生可执行文件, 直接调用操作系统内置的 WebView2 渲染内核, 用户无需单独安装浏览器, 但需确保系统 WebView2 Runtime 处于最新状态。数据库采用 SQLite 3, 以单文件形式嵌入应用数据目录, 无需配置独立的数据库服务器。

1.3 技术栈架构

系统采用了前后端分离但本地集成的架构, 旨在结合 Web 开发的灵活性与原生应用的高性能。前端层基于 Vue 3 框架构建, 使用 Naive UI 组件库打造现代化用户界面, 利用 Pinia 进行全局状态管理, 并通过 Vue Router 处理页面路由逻辑。后端层构建于 Tauri 2 框架之上, 利用 Rust 语言处理系统调用、文件 I/O 以及与 SQLite 数据库的交互, 确保了业务逻辑的安全性与执行效率。数据层选用 SQLite 3 关系数据库, 通过 Rust 的 `rusqlite` 库进行连接和操作, 保证了数据存储的 ACID 特性。算法层集成了 Google OR-Tools 约束编程-可满足性求解器 (CP-SAT), 核心逻辑由 Python 编写并使用 PyInstaller 打包为独立二进制文件, 由 Rust 主进程以 Sidecar 的形式进行调用和管理。

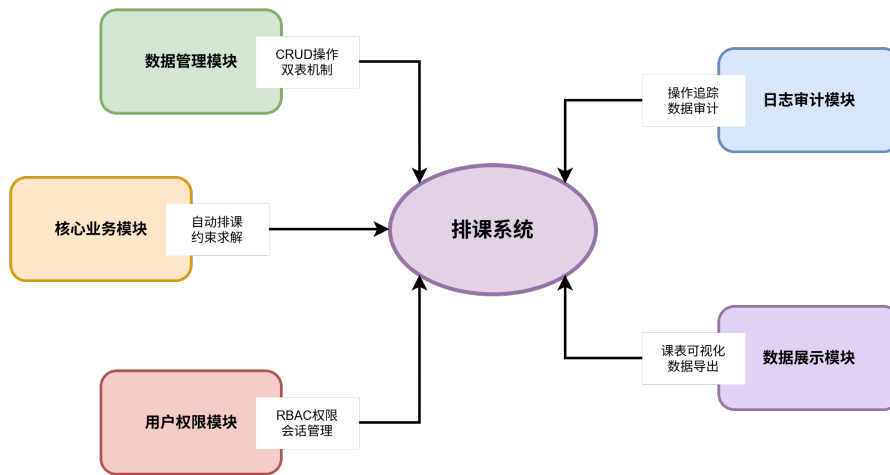


图 1 系统功能结构图

二、系统功能结构

本系统依据业务逻辑划分为五个核心模块，各模块之间通过统一的数据总线进行交互，共同支撑起排课系统的完整流程。

1. **数据管理模块**：负责基础数据的增删改查（CRUD）。包含教师信息管理、课程信息管理、场地与校区管理、时间段与工作日配置。该模块实现了“双表机制”，即用户操作先写入临时表，确认无误后才提交至主表，有效防止误操作。
2. **核心业务模块**：自动排课功能是系统的智能核心。负责提取当前的基础数据，序列化为 JSON 格式传递给 Python 求解器，求解器根据硬约束（如冲突检测）和软约束（如排课密度优化）计算出最优课表，并将结果返回给前端进行渲染。
3. **用户权限模块**：实现基于角色（RBAC）的安全控制。系统预设了 Scheduler（排课员）和 Teacher（教师）两种角色。排课员拥有全系统权限，而教师仅能查看与导出个人课表。该模块还包含用户注册、密码重置及会话管理功能。
4. **日志审计模块**：构建了细粒度的操作追踪体系。系统自动记录所有关键的数据变更（如新增、修改、删除、提交、回滚），并记录操作人、操作时间及具体的字段变更差异（Diff），为数据安全提供追溯能力。
5. **数据展示模块**：提供可视化的课表视图。包括“校区总课表”和“教师个人课表”。支持按校区、场地筛选查看，并提供将课表导出为 CSV 文件的功能。

三、基本表定义与完整性约束

数据库设计严格遵循第三范式 (3NF), 消除了数据冗余和传递依赖。为了实现“编辑-提交-回滚”的乐观事务特性, 系统采用了**双表架构**。除系统配置类表(如 `roles`, `users`) 外, 所有核心业务数据表(`teachers`, `courses`, `campuses`, `venues`, `time_slots`, `days`, `scheduled_classes` 等) 均拥有结构完全相同的临时表(后缀为 `_temp`)。

这种设计使得用户的编辑操作(如排课、修改信息)首先作用于临时表, `audit_logs_temp` 用于记录这些未提交操作产生的日志。只有当用户执行“提交”操作时, 临时表的数据才会同步至主表, 同时相关的临时审计日志也会转入主表。而对于用户管理、密码重置、登录注销等立即生效的系统级操作, 其产生的日志则直接写入主表 `audit_logs`, 不经过临时表流转。

3.1 基本表定义

系统包含以下核心实体表及其主外码定义:

基础信息表:

- `campuses`: 存储校区信息。主码为 `id` (TEXT), 无外码。
- `venues`: 存储场地信息。主码为 `id` (TEXT), 外码 `campus_id` 引用 `campuses(id)`。
- `courses`: 存储课程基本信息。主码为 `id` (TEXT), 无外码。
- `teachers`: 存储教师信息。主码为 `id` (TEXT), 无外码。
- `time_slots` 与 `days`: 存储排课的时间维度信息。主码分别为 `id` (TEXT), 无外码。

关系表 (多对多关联):

- `course_venues`: 定义课程与其允许上课场地的关系。主码为复合键 (`course_id`, `venue_id`), 分别引用 `courses(id)` 和 `venues(id)`。
- `teacher_courses`: 定义教师与其授课资质的关系。主码为复合键 (`teacher_id`, `course_id`), 分别引用 `teachers(id)` 和 `courses(id)`。
- `teacher_unavailability`: 记录教师的不可用时间。主码为复合键 (`teacher_id`, `day_id`, `time_id`), 分别引用 `teachers(id)`, `days(id)` 和 `time_slots(id)`。

核心业务表:

- `scheduled_classes`: 存储最终的排课结果。主码为 `id` (TEXT), 包含外码 `teacher_id`, `course_id`, `day_id`, `time_id`, `campus_id`, `venue_id` 分别引用对应的基础表主码。
- `schedule_density`: 存储排课密度约束。主码为复合键 (`campus_id`, `day_id`, `time_id`), 引用校区和时间表。

系统管理表:

- `users`: 存储用户信息。主码为 `id` (TEXT), 外码 `role_id` 引用 `roles(id)`, `teacher_id` 引用 `teachers(id)`。
- `roles`: 定义角色权限。主码为 `id` (TEXT)。
- `audit_logs`: 存储审计日志。主码为 `id` (TEXT), 外码 `user_id` 引用 `users(id)`。

3.2 完整性约束定义

系统通过数据库定义语言 (DDL) 严格实施了实体完整性和参照完整性:

主键约束: 所有表均定义了 `PRIMARY KEY`。实体表使用 UUID 字符串作为单属性主键 (如 `id TEXT PRIMARY KEY`), 关系表使用复合主键 (如 `PRIMARY KEY (course_id, venue_id)`) 以确保关系的唯一性。

外键约束: 系统显式开启了外键检查 (`PRAGMA foreign_keys = ON`)。所有关联字段均定义了外键约束, 并广泛使用了级联删除 (`ON DELETE CASCADE`) 策略。例如, `venues` 表中的 `campus_id` 引用 `campuses(id)`, 当删除一个校区时, 数据库会自动级联删除该校区下的所有场地, 进而级联删除相关的课程关联和排课记录, 彻底杜绝了孤儿数据的产生, 保证了数据的一致性。

3.3 索引定义

为了优化查询性能, 系统在常用的外键列和查询条件列上建立了索引。除了主键自动生成的唯一索引外, 还显式创建了以下索引:

- 针对关联查询: `idx_venues_campus_id`、`idx_course_venues_course_id` 等, 加速多表连接操作。
- 针对业务查询: `idx_audit_timestamp` 加速日志按时间倒序查询, `idx_users_username` 保证用户名唯一性查询的效率。

四、系统安全性设计

系统在安全性方面采用了多层防御策略, 涵盖了用户认证、权限控制及数据追踪。

4.1 用户认证与会话管理

系统不存储明文密码, 而是通过 Rust 后端的 `auth.rs` 模块使用 `bcrypt` 算法对密码进行加盐哈希存储。用户登录时, 系统比对哈希值验证身份。为了防止并发操作导致的数据冲突, 系统实施了严格的单会话限制: 在用户登录成功后, 会在本地创建一个文件锁 (Session Lock), 如果检测到锁文件已被占用, 则拒绝新的登录请求, 确保同一时间只有一个活跃会话操作数据。

4.2 外模式与权限控制

系统通过 `roles` 表定义了不同人员的外模式权限:

- **排课员 (Scheduler):** 拥有最高权限。可以访问所有数据管理页面, 执行自动排课算法, 执行数据的提交与回滚, 管理用户账号, 以及查看所有审计日志。其外模式覆盖了数据库中的所有表。
- **教师 (Teacher):** 权限受限。登录后仅能访问“教师个人课表”页面和“设置”页面。后端通过 `get_current_user` 接口返回的 `teacher_id` 进行数据过滤, 确保教师只能查询 `scheduled_classes` 表中与自己 ID 相关的记录, 实现了行级的数据隔离。

4.3 审计与追踪

为了满足安全性审计需求, 系统内置了强制日志记录机制。针对不同类型的操作, 系统采用了差异化的日志生成策略: 对于复杂的业务数据变更 (如排课信息的修改、基础数据的更新), 前端利用 Pinia 状态管理中的 `jsComputeDiff` 算法计算出新旧数据的详细差异 (Diff), 生成 JSON

格式的变更详情后传输给后端存储;而对于用户重置密码、登录登出等敏感或简单的系统操作,则直接由 Rust 后端在执行操作时生成日志条目。这些日志统一存储在 `audit_logs` 表中,排课员可随时查阅,确保了系统操作的不可抵赖性。

五、应用层数据处理逻辑

由于本系统采用 SQLite 嵌入式数据库,且业务逻辑高度依赖于 Rust 后端的强类型处理,因此并未在数据库层面编写传统的 SQL 存储过程或触发器,而是采用“应用层事务脚本”模式,在 Rust 代码中实现了等效的数据处理逻辑。

5.1 数据提交过程

`db_handler.rs` 中的 `commit_data` 函数相当于数据库的存储过程。该函数在一个原子事务中执行复杂的逻辑:

1. 首先获取当前会话的用户 ID 用于审计。
2. 开启数据库事务。
3. 执行一系列 SQL 语句,将所有 `_temp` 表的数据同步到主表。这涉及先删除主表中不在临时表的数据,更新已存在的数据,再插入新增的数据。
4. 自动检测新添加的教师,并调用 `auto_create_user_for_teacher` 函数为新教师自动生成登录账号。
5. 记录一次 `COMMIT_OPERATION` 的审计日志。
6. 提交事务,清空临时表。

这一过程保证了数据从“编辑态”到“发布态”转换的原子性和一致性。

5.2 自动账户生成逻辑

`db_handler.rs` 中的 `auto_create_user_for_teacher` 函数充当了业务层面的触发器。当管理员添加新教师并提交时,该函数会被触发:

1. 检查该教师是否已存在关联账号。
2. 若不存在,则基于教师姓名生成唯一的用户名(如遇重名自动追加数字后缀)。
3. 生成默认密码的哈希值。
4. 在 `users` 表中插入新记录,并将角色设定为“Teacher”。

这一逻辑确保了业务规则的自动化执行,减轻了管理员的手工维护成本。

六、主要技术与主要模块

6.1 主要技术

6.1.1 Rust 与 Tauri 的跨语言互操作

系统利用 Tauri 的 IPC（进程间通信）机制实现了前端 JavaScript 与后端 Rust 的无缝交互。前端通过 `invoke` 命令异步调用后端函数，Rust 后端使用 `serde` 库将复杂的数据结构（如 `AllData` 结构体）自动序列化为 JSON 传递给前端。这种设计既利用了 Rust 处理数据库和文件 I/O 的高性能，又保留了 Web 前端开发的灵活性。

6.1.2 OR-Tools 约束求解技术

排课问题的核心是解决资源冲突。系统集成了 Google OR-Tools 的 CP-SAT 求解器。在 Python 脚本中，系统建立了数学模型，定义了决策变量（如某教师在某天某时某地是否上课），并添加了硬约束（如教师时间不冲突、场地容量限制）和软约束（如最小化单课日、均衡校区分布）。通过定义目标函数并最小化惩罚值，求解器能够在巨大的搜索空间中快速找到最优或近似最优的排课方案。

6.1.3 乐观双表机制

为了解决排课过程中频繁试错的需求，系统设计了独特的“双表机制”。用户在前端的所有编辑操作都会通过防抖函数（Debounce）自动保存到 SQLite 的 `_temp` 临时表中。此时主表数据不受影响，保证了系统的稳定运行。只有当用户点击“提交”时，临时表数据才会覆盖主表。这种机制结合事务回滚功能，为用户提供了一个安全的“沙箱”环境，可以随意尝试不同的排课策略而不必担心破坏现有数据。

6.2 主要模块论述

6.2.1 数据处理模块 `db_handler.rs`

这是后端的骨架，封装了 `rusqlite` 连接池（`AppState`）。它负责数据库的初始化（执行 `schema.sql`）、数据的加载与保存、事务管理以及审计日志的记录。该模块严格保证了所有写操作都在事务中执行，确保了 ACID 特性。

6.2.2 排课求解器模块 `solver.py`

这是系统的“大脑”。它包含 `DataManager` 类用于预处理数据和建立索引映射，以及 `solve_scheduling` 函数用于构建约束模型。该模块独立于主程序运行，通过标准输入输出与 Rust 主进程交换 JSON 数据，实现了计算密集型任务与 IO 密集型任务的分离。

6.2.3 前端状态管理模块 `stores/data.js`

基于 Pinia 构建，维护了全局的应用状态。它不仅存储了从后端加载的数据，还实现了智能的差异对比算法（`jsComputeDiff`），用于在前端生成精细的审计日志变更详情。同时，它通过 `watch` 监听数据变化，实现了自动保存到后端临时表的逻辑。

七、系统功能运行实例

7.1 登录与角色分流

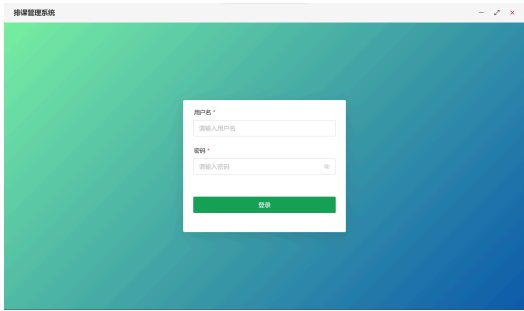
系统启动后进入登录界面。为了方便系统初始化与测试,系统预设了默认排课员账号 `admin`, 密码为 `123456`。用户输入账号密码, 后端验证通过后返回会话 ID 及角色信息。若以排课员账号登录, 系统自动跳转至“校区总课表”页面, 并在侧边栏显示所有管理菜单(教师管理、课程管理、自动排课等)。若以教师账号登录, 系统自动跳转至“教师个人课表”页面, 侧边栏仅显示查看类菜单, 确保权限隔离。

7.2 数据录入与提交

管理员进入“课程管理”页面, 点击“新增课程”, 输入课程名称并指定可选的校区和场地。操作完成后, 界面右上角会自动显示“保存更改”按钮变为可用状态。此时数据仅存在于临时表中。管理员点击“保存更改”按钮, 系统弹出确认框, 确认后后端执行事务提交, 数据正式写入主表, 并生成一条 `COMMIT_OPERATION` 的审计日志。

7.3 自动排课流程

管理员点击侧边栏的“自动排课”按钮。前端显示加载动画, 后端将当前所有相关数据(教师、课程、场地、约束)打包为 JSON, 启动 Python 求解器进程。等待一段时间后, 求解器返回排课结果。系统自动将新生成的课表加载到临时视图中。管理员可以在“校区总课表”中预览排课效果, 此时若不满意, 可直接点击“撤销所有更改”, 系统瞬间恢复到排课前的状态。

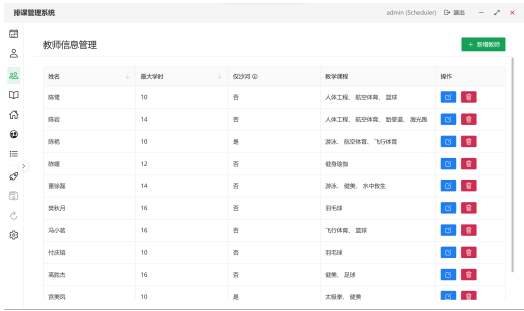


(a) 用户系统登录界面

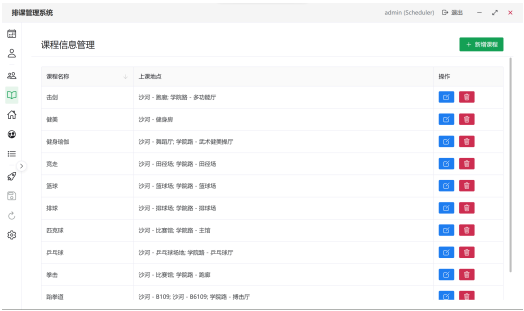


(b) 教师账号登录后界面

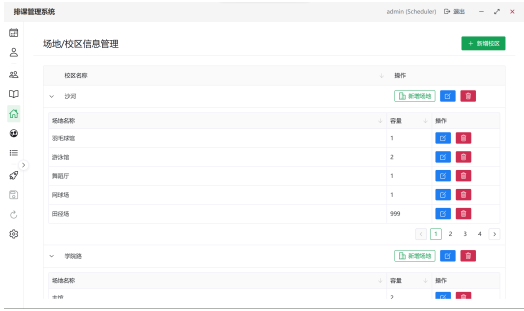
表 1 系统登录与初始化界面



(a) 教师信息管理



(b) 课程信息管理



(c) 场地信息管理

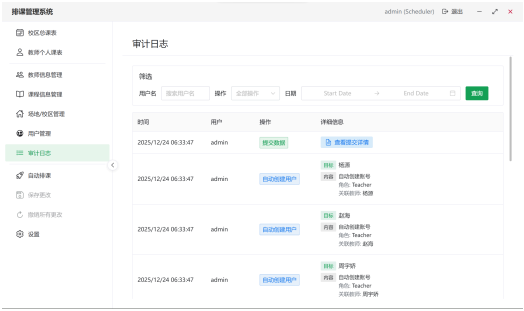


(d) 自动排课结果

表 2 系统基础数据管理界面



(a) 数据导入导出



(b) 为导入的教师自动创建用户

表 3 系统设置与审计界面

八、源程序简要说明

源代码采用模块化组织, 结构清晰:

- `src-tauri/schema.sql`: 数据库定义的源头, 包含主表、临时表、索引及初始化数据的 SQL 语句。
- `src-tauri/src/models.rs`: 定义了 Rust 侧的数据结构, 与数据库表一一对应, 并实现了 `Serialize` 和 `Deserialize` trait, 是前后端数据交互的契约。
- `src-tauri/src/db_handler.rs`: 核心数据库逻辑, 实现了 `commit_temp_to_main` 等关键事务函数。
- `src-tauri/src/auth.rs` & `audit.rs`: 分别负责密码安全验证和审计日志的结构化记录。
- `src/stores/data.js`: 前端的数据仓库, 实现了数据的响应式更新、自动保存逻辑以及变更差异计算。
- `solver/solver.py`: 独立的 Python 脚本, 包含了使用 OR-Tools 进行约束建模的全部逻辑。

```
course-scheduler/
├── src/
│   ├── # Vue 前端源码
│   │   ├── pages/
│   │   │   ├── # 页面组件
│   │   │   ├── CourseManagement.vue
│   │   │   ├── TeacherManagement.vue
│   │   │   ├── VenueManagement.vue
│   │   │   ├── TeacherTimetable.vue
│   │   │   └── CampusTimetable.vue
│   │   ├── stores/
│   │   │   ├── # Pinia 状态管理
│   │   │   └── data.js
│   │   ├── components/
│   │   │   ├── # 可复用组件
│   │   │   └── layouts/
│   │   │       ├── # 布局组件
│   │   │       └──
│   ├── # Rust 后端源码
│   │   ├── src/
│   │   │   ├── main.rs
│   │   │   ├── # Tauri 主入口
│   │   │   ├── models.rs
│   │   │   ├── # 数据模型定义
│   │   │   ├── db_handler.rs
│   │   │   ├── # SQLite 数据库操作
│   │   │   ├── import_export.rs
│   │   │   ├── # 导入导出功能
│   │   │   └── lib.rs
│   │   │       ├── # 库入口
│   │   │       ├── schema.sql
│   │   │       ├── # 数据库表结构定义
│   │   │       └── tauri.conf.json
│   │           ├── # Tauri 配置
│   ├── solver/
│   │   ├── # Python 排课求解器
│   │   ├── solver.py
│   │   ├── # OR-Tools 约束模型
│   │   └── solver.spec
│   │       ├── # PyInstaller 打包配置
│   └── scripts/
│       └── prepare-solver.js
│           ├── # 求解器构建脚本
```

九、收获和体会

本次课程排课系统的开发是一次将数据库理论与现代软件工程实践深度结合的宝贵经历。

首先, 我们深刻理解了**数据库规范化设计**的重要性。在设计初期, 严格遵循 3NF 将课程、场地、教师及其关系拆分为独立的实体表和关联表, 虽然增加了表连接的查询复杂度, 但在后续开发中极大地简化了数据维护逻辑, 避免了更新异常和数据不一致问题, 特别是在处理级联删除时表现尤为出色。

其次, **双表架构**的设计是本项目的一大创新点。通过引入临时表和事务机制, 巧妙地解决了长事务(用户长时间编辑)与数据库短事务之间的矛盾。这种设计模式不仅提升了用户体验(支持撤销、自动保存), 也从根本上保证了核心业务数据的安全性, 体现了数据库事务(Transaction)在实际应用场景中的灵活运用。

最后, 通过整合 Rust、Python 和 Web 技术, 我们体会到了**多语言混合编程**的优势。利用 Rust 处理高性能 I/O 和类型安全, Python 处理复杂的数学建模, Vue 处理动态交互, 各取所长。这一过程加深了对系统架构设计、接口契约定义以及数据流转控制的理解, 提升了解决复杂工程问题的综合能力。